DEEP LEARNING – MOD 1. FROM MACHINE LEARNING TO DEEP LEARNING (THEORY)

SAMPLE VS POPULATION

A population refers to the entire set of individuals, objects, or data points that you want to study. It can be large or small depending on the scope of your research. For example, all students in a school or all people in a country.

A sample is a subset of the population that is selected for analysis. It's used when studying the entire population is impractical or impossible. Sampling allows for inferences about the population using statistical techniques.

The population gives a complete picture, while the sample provides an estimate. Parameters (like population mean) describe the population; statistics (like sample mean) describe the sample. The population refers to the entire group of individuals or items that we are interested in studying and drawing conclusions about. In statistics, the population is the entire set of items from which data is drawn in the statistical study. It can be a group of individuals or a set of items. The population is usually denoted by **N.**

A sample is a subset of the population selected for study. It is a representative portion of the population from which we collect data in order to make inferences or draw conclusions about the entire population. The sample is denoted by **n.**

Population vs Sample

Population	**Sample**
The population includes all members of a specified group.	A sample is a subset of the population.
Collecting data from an entire population can be time-consuming, expensive, and sometimes impractical or impossible.	Samples offer a more feasible approach to studying populations, allowing researchers to draw conclusions based on smaller, manageable datasets
Includes all residents in the city.	Consists of 1000 households, a subset of the entire population.

Collecting Data From Population and Sample

Populations are used when your research question requires, or when you have access to, data from every member of the population. Usually, it is only straightforward to collect data from a whole population when it is small, accessible and cooperative.

Example:

A marketing manager for a small local bakery wants to understand customer preferences for different types of bread sold at their shop. Since they are solely interested in analyzing the preferences of customers who visit their bakery, they decide to collect data on bread preferences from every

customer who makes a purchase over the course of a month. By using the entire dataset of bread purchases, including preferences indicated by customers, they aim to identify trends and patterns in bread choices specifically among their bakery's clientele.

When your population is large in size, geographically dispersed, or difficult to contact, it's necessary to use a sample. With statistical analysis, you can use sample data to make estimates or test hypotheses about population data.

Example:

Suppose you are conducting research on smartphone usage habits among teenagers in a specific city. Your population comprises all teenagers aged 13-18 living in that city, which could number in the tens of thousands. Due to logistical constraints and the difficulty of reaching every teenager in the city, you opt to use a sample of 500 teenagers randomly selected from different schools within the city. This sample will participate in interviews or surveys to provide insights into their smartphone usage patterns, preferences, and behaviors.

When Should Samples be used?

- When studying a large population where it is impractical or impossible to collect data from every individual.
- When resources such as time, cost, and manpower are limited, making it more feasible to collect data from a subset of the population.
- When conducting research or experiments where it is important to minimize potential biases in data collection.

Population Parameter vs Sample Statistic

Population Parameter	Sample Statistic
It is a numerical characteristic that describes the entire population	Statistics are calculated from sample data and serve as estimates or approximations of the corresponding population parameters
Parameters are typically unknown and must be estimated.	Calculated using data from a sample drawn from the population. Statistics are directly computed from sample data.
Calculated using data from a sample drawn from the population. Statistics are directly computed from sample data.	Used to estimate population parameters based on sample data. Statistics help researchers infer population characteristics from a representative subset of the population

Example:
When examining the height of adult males in a country, the population parameter represents the population mean height (μ), which reflects the average height of all adult males nationwide. However, due to the impracticality of measuring the length of each individual, this parameter remains unknown and must be estimated. In contrast, a sample Statisticssuch as the sample mean ($\bar{x}$) is derived from a subset of the population, in this case a random sample of 500 male adults from different regions of the country. By measuring the heights of these individuals and averaging

them, researchers obtain a sample Statistics that serves as an approximate population parameter. This process makes it possible to draw conclusions about the height distribution of adult men in the country, based on the characteristics observed in the sample.

SUPERVISED LEARNING

Supervised machine learning is a fundamental approach for machine learning and artificial intelligence. It involves training a model using labeled data, where each input comes with a corresponding correct output. The process is like a teacher guiding a student—hence the term "supervised" learning.

supervised learning is a type of machine learning where a model is trained on labeled data—meaning each input is paired with the correct output. the model learns by comparing its predictions with the actual answers provided in the training data. Over time, it adjusts itself to minimize errors and improve accuracy. The goal of supervised learning is to make accurate predictions when given new, unseen data. For example, if a model is trained to recognize handwritten digits, it will use what it learned to correctly identify new numbers it hasn't seen before.

Supervised learning can be applied in various forms, including supervised learning classification and supervised learning regression, making it a crucial technique in the field of artificial intelligence and supervised data mining.

A fundamental concept in supervised machine learning is learning a class from examples. This involves providing the model with examples where the correct label is known, such as learning to classify images of cats and dogs by being shown labeled examples of both. The model then learns the distinguishing features of each class and applies this knowledge to classify new images.

How Supervised Machine Learning Works?

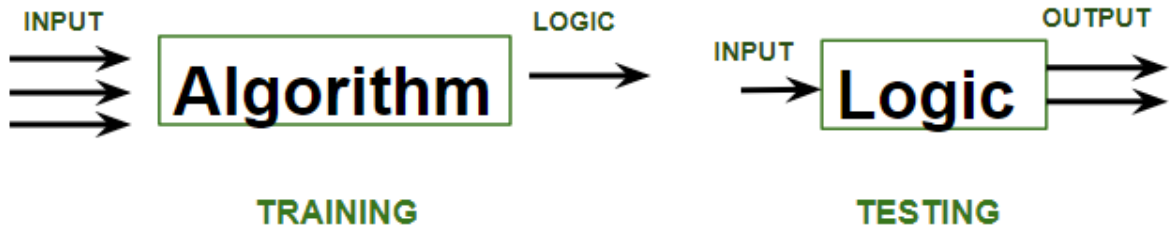
Where **supervised learning algorithm** consists of input features and corresponding output labels. The process works through:

- **Training Data:** The model is provided with a training dataset that includes input data (features) and corresponding output data (labels or target variables).
- **Learning Process:** The algorithm processes the training data, learning the relationships between the input features and the output labels. This is achieved by adjusting the model's parameters to minimize the difference between its predictions and the actual labels.

After training, the model is evaluated using a test dataset to measure its accuracy and performance. Then the model's performance is optimized by adjusting parameters and using techniques like cross-validation to balance bias and variance. This ensures the model generalizes well to new, unseen data.

In summary, supervised machine learning involves training a model on labeled data to learn patterns and relationships, which it then uses to make accurate predictions on new data.

Let's learn how a supervised machine learning model is trained on a dataset to learn a mapping function between input and output, and then with learned function is used to make predictions on new data:



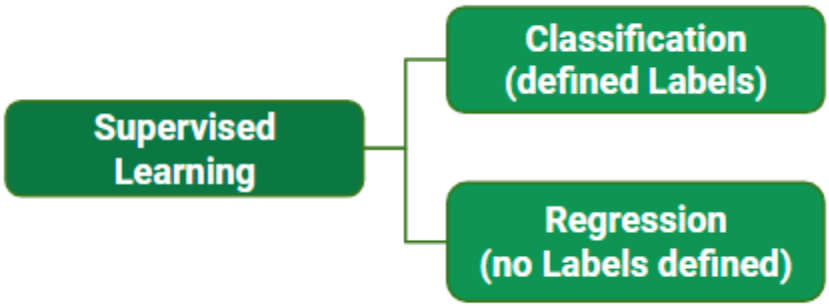
In the image above,

- **Training** phase involves feeding the algorithm labeled data, where each data point is paired with its correct output. The algorithm learns to identify patterns and relationships between the input and output data.
- **Testing** phase involves feeding the algorithm new, unseen data and evaluating its ability to predict the correct output based on the learned patterns.

Types of Supervised Learning in Machine Learning

Now, Supervised learning can be applied to two main types of problems:

- Classification: Where the output is a categorical variable (e.g., spam vs. non-spam emails, yes vs. no).
- Regression: Where the output is a continuous variable (e.g., predicting house prices, stock prices).



While training the model, data is usually split in the ratio of 80:20 i.e. 80% as training data and the rest as testing data. In training data, we feed input as well as output for 80% of data. The model learns from training data only. We use different **supervised learning algorithms** (which we will discuss in detail in the next section) to build our model. Let's first understand the classification and regression data through the table below:

User ID	Gender	Age	Salary	Purchased
15624510	Male	19	19000	0
15810944	Male	35	20000	1
15668575	Female	26	43000	0
15603246	Female	27	57000	0
15804002	Male	19	76000	1
15728773	Male	27	58000	1
15598044	Female	27	84000	0
15694829	Female	32	150000	1
15600575	Male	25	33000	1
15727311	Female	35	65000	0
15570769	Female	26	80000	1
15606274	Female	26	52000	0
15746139	Male	20	86000	1
15704987	Male	32	18000	0
15628972	Male	18	82000	0
15697686	Male	29	80000	0
15733883	Male	47	25000	1

Figure A: CLASSIFICATION

Temperature	Pressure	Relative Humidity	Wind Direction	Wind Speed
10.69261758	986.882019	54.19337313	195.7150879	3.278597116
13.59184184	987.8729248	48.0648859	189.2951202	2.909167767
17.70494885	988.1119385	39.11965597	192.9273834	2.973036289
20.95430404	987.8500366	30.66273218	202.0752869	2.965289593
22.9278274	987.2833862	26.06723423	210.6589203	2.798230886
24.04233986	986.2907104	23.46918024	221.1188507	2.627005816
24.41475295	985.2338867	22.25082295	233.7911987	2.448749781
23.93361956	984.8914795	22.35178837	244.3504333	2.454271793
22.68800023	984.8461304	23.7538641	253.0864716	2.418341875
20.56425726	984.8380737	27.07867944	264.5071106	2.318677425
17.76400389	985.4262085	33.54900114	280.7827454	2.343950987
11.25680746	988.9386597	53.74139903	68.15406036	1.650191426
14.37810685	989.6819458	40.70884681	72.62069702	1.553469896
18.45114201	990.2960205	30.85038484	71.70604706	1.005017161
22.54895853	989.9562988	22.81738811	44.66042709	0.264133632
24.23155922	988.796875	19.74790765	318.3214111	0.329656571

Figure B: REGRESSION

Both the above figures have labelled data set as follows:

- **Figure A:** It is a dataset of a shopping store that is useful in predicting whether a customer will purchase a particular product under consideration or not based on his/ her gender, age, and salary.
  Input: Gender, Age, Salary
  Output: Purchased i.e. 0 or 1; 1 means yes the customer will purchase and 0 means that the customer won't purchase it.

- **Figure B:** It is a Meteorological dataset that serves the purpose of predicting wind speed based on different parameters.
  Input: Dew Point, Temperature, Pressure, Relative Humidity, Wind Direction
  Output: Wind Speed

Practical Examples of Supervised learning

Few practical examples of supervised machine learning across various industries:

- **Fraud Detection in Banking:** Utilizes supervised learning algorithms on historical transaction data, training models with labeled datasets of legitimate and fraudulent transactions to accurately predict fraud patterns.

- **Parkinson Disease Prediction:** Parkinson's disease is a progressive disorder that affects the nervous system and the parts of the body controlled by the nerves.

- **Customer Churn Prediction:** Uses supervised learning techniques to analyze historical customer data, identifying features associated with churn rates to predict customer retention effectively.

- **Cancer cell classification:** Implements supervised learning for cancer cells based on their features, and identifying them if they are 'malignant' or 'benign.

- **Stock Price Prediction:** Applies supervised learning to predict a signal that indicates whether buying a particular stock will be helpful or not.

Supervised Machine Learning Algorithms

Supervised learning can be further divided into several different types, each with its own unique characteristics and applications. Here are some of the most common types of supervised learning algorithms:

- **Linear Regression**: Linear regression is a type of supervised learning regression algorithm that is used to predict a continuous output value. It is one of the simplest and most widely used algorithms in supervised learning.
- **Logistic Regression** : Logistic regression is a type of supervised learning classification algorithm that is used to predict a binary output variable.
- **Decision Trees** : Decision tree is a tree-like structure that is used to model decisions and their possible consequences. Each internal node in the tree represents a decision, while each leaf node represents a possible outcome.
- **Random Forests** : Random forests again are made up of multiple decision trees that work together to make predictions. Each tree in the forest is trained on a different subset of the input features and data. The final prediction is made by aggregating the predictions of all the trees in the forest.
- **Support Vector Machine(SVM)** : The SVM algorithm creates a hyperplane to segregate n-dimensional space into classes and identify the correct category of new data points. The extreme cases that help create the hyperplane are called support vectors, hence the name Support Vector Machine.
- **K-Nearest Neighbors (KNN) :** KNN works by finding k training examples closest to a given input and then predicts the class or value based on the majority class or average value of these neighbors. The performance of KNN can be influenced by the choice of k and the distance metric used to measure proximity.
- **Gradient Boosting** : Gradient Boosting combines weak learners, like decision trees, to create a strong model. It iteratively builds new models that correct errors made by previous ones.
- **Naive Bayes Algorithm**: The Naive Bayes algorithm is a supervised machine learning algorithm based on applying Bayes' Theorem with the "naive" assumption that features are independent of each other given the class label.

Let's summarize the **supervised machine learning algorithms** in table:

Algorithm	**Regression, Classification**	**Purpose**	**Method**	**Use Cases**
Linear Regression	Regression	Predict continuous output values	Linear equation minimizing sum of squares of residuals	Predicting continuous values

Algorithm	Regression, Classification	Purpose	Method	Use Cases
Logistic Regression	Classification	Predict binary output variable	Logistic function transforming linear relationship	Binary classification tasks
Decision Trees	Both	Model decisions and outcomes	Tree-like structure with decisions and outcomes	Classification and Regression tasks
Random Forests	Both	Improve classification and regression accuracy	Combining multiple decision trees	Reducing overfitting, improving prediction accuracy
SVM	Both	Create hyperplane for classification or predict continuous values	Maximizing margin between classes or predicting continuous values	Classification and Regression tasks
KNN	Both	Predict class or value based on k closest neighbors	Finding k closest neighbors and predicting based on majority or average	Classification and Regression tasks, sensitive to noisy data
Gradient Boosting	Both	Combine weak learners to create strong model	Iteratively correcting errors with new models	Classification and Regression tasks to improve prediction accuracy
Naive Bayes	Classification	Predict class based on feature independence assumption	Bayes' theorem with feature independence assumption	Text classification, spam filtering, sentiment analysis, medical

These **types of supervised learning in machine learning** vary based on the problem you're trying to solve and the dataset you're working with. In classification problems, the task is to assign inputs to predefined classes, while regression problems involve predicting numerical outcomes.

Training a Supervised Learning Model: Key Steps

The goal of Supervised learning is to generalize well to unseen data. Training a model for supervised learning involves several crucial steps, each designed to prepare the model to make accurate predictions or decisions based on labeled data. Below are the key steps involved in training a model for supervised machine learning:

1. **Data Collection and Preprocessing**: Gather a labeled dataset consisting of input features and target output labels. Clean the data, handle missing values, and scale features as needed to ensure high quality for **supervised learning** algorithms.
2. **Splitting the Data**: Divide the data into **training set** (80%) and the **test set** (20%).
3. **Choosing the Model**: Select appropriate algorithms based on the problem type. This step is crucial for effective **supervised learning** in AI.
4. **Training the Model**: Feed the model input data and output labels, allowing it to learn patterns by adjusting internal parameters.
5. **Evaluating the Model**: Test the trained model on the unseen test set and assess its performance using various metrics.
6. **Hyperparameter Tuning:** Adjust settings that control the training process (e.g., learning rate) using techniques like grid search and cross-validation.
7. **Final Model Selection and Testing**: Retrain the model on the complete dataset using the best hyperparameters testing its performance on the test set to ensure readiness for deployment.
8. **Model Deployment**: Deploy the validated model to make predictions on new, unseen data.

By following these steps, supervised learning models can be effectively trained to tackle various tasks, from **learning a class from examples** to making predictions in real-world applications.

Advantages and Disadvantages of Supervised Learning

Advantages of Supervised Learning

The power of **supervised learning** lies in its ability to accurately predict patterns and make data-driven decisions across a variety of applications. Here are some advantages of **supervised learning** listed below:

- **Supervised learning** excels in accurately predicting patterns and making data-driven decisions.
- **Labeled training data** is crucial for enabling **supervised learning models** to learn input-output relationships effectively.
- **Supervised machine learning** encompasses tasks such as **supervised learning classification** and **supervised learning regression**.
- Applications include complex problems like image recognition and natural language processing.
- Established evaluation metrics (accuracy, precision, recall, F1-score) are essential for assessing **supervised learning model** performance.

- Advantages of **supervised learning** include creating complex models for accurate predictions on new data.
- **Supervised learning** requires substantial labeled training data, and its effectiveness hinges on data quality and representativeness.

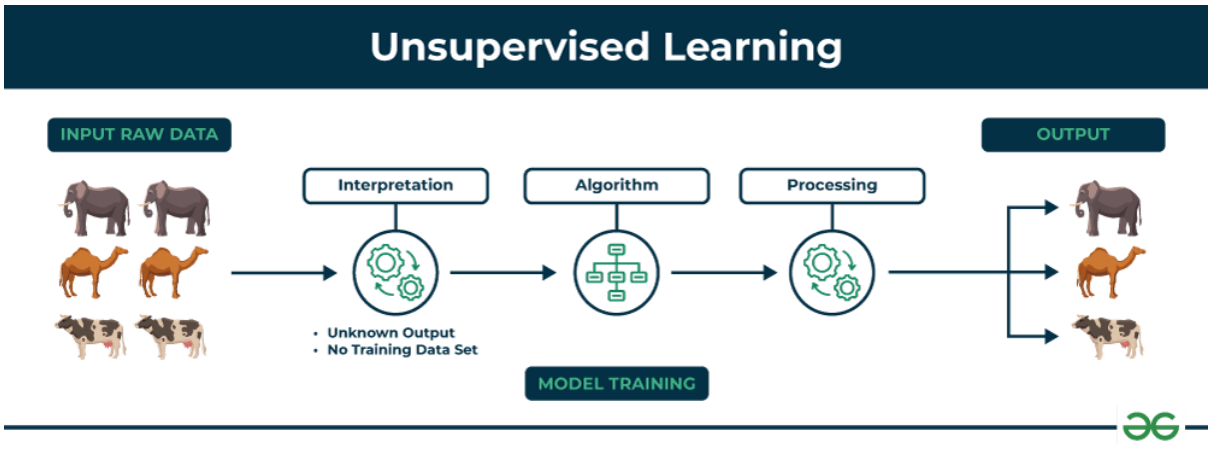
Disadvantages of Supervised Learning

Despite the benefits of **supervised learning methods**, there are notable **disadvantages of supervised learning**:

1. **Overfitting**: Models can overfit training data, leading to poor performance on new data due to capturing noise in **supervised machine learning**.
2. **Feature Engineering** : Extracting relevant features is crucial but can be time-consuming and requires domain expertise in **supervised learning applications**.
3. **Bias in Models:** Bias in the training data may result in unfair predictions in **supervised learning algorithms**.
4. **Dependence on Labeled Data**: **Supervised learning** relies heavily on labeled training data, which can be costly and time-consuming to obtain, posing a challenge for **supervised learning techniques**.

UNSUPERVISED LEARNING

Unsupervised learning is a branch of machine learning that deals with unlabeled data. Unlike supervised learning, where the data is labeled with a specific category or outcome, unsupervised learning algorithms **are tasked with finding patterns and relationships within the data without any prior knowledge of the data's meaning**. Unsupervised machine learning algorithms **find hidden patterns and data without any human intervention, i.e., we don't give output to our model. The training model has only input parameter values and discovers the groups or patterns on its own.**



Unsupervised Learning

The image shows set of animals: elephants, camels, and cows that represents raw data that the unsupervised learning algorithm will process.

- The "Interpretation" stage signifies that the algorithm doesn't have predefined labels or categories for the data. It needs to figure out how to group or organize the data based on inherent patterns.

- **Algorithm** represents the core of unsupervised learning process using techniques like clustering, dimensionality reduction, or anomaly detection to identify patterns and structures in the data.
- **Processing** stage shows the algorithm working on the data.

The output shows the results of the unsupervised learning process. In this case, the algorithm might have grouped the animals into clusters based on their species (elephants, camels, cows).

How does unsupervised learning work?

Unsupervised learning works by analyzing unlabeled data to identify patterns and relationships. The data is not labeled with any predefined categories or outcomes, so the algorithm must find these patterns and relationships on its own. This can be a challenging task, but it can also be very rewarding, as it can reveal insights into the data that would not be apparent from a labeled dataset.

Data-set in Figure A is Mall data that contains information about its clients that subscribe to them. Once subscribed they are provided a membership card and the mall has complete information about the customer and his/her every purchase. Now using this data and unsupervised learning techniques, the mall can easily group clients based on the parameters we are feeding in.

CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
1	Male	19	15	39
2	Male	21	15	81
3	Female	20	16	6
4	Female	23	16	77
5	Female	31	17	40
6	Female	22	17	76
7	Female	35	18	6
8	Female	23	18	94
9	Male	64	19	3
10	Female	30	19	72
11	Male	67	19	14
12	Female	35	19	99
13	Female	58	20	15
14	Female	24	20	77
15	Male	37	20	13
16	Male	22	20	79
17	Female	35	21	35

Figure A

The input to the unsupervised learning models is as follows:

- **Unstructured data**: May contain noisy(meaningless) data, missing values, or unknown data
- **Unlabeled data**: Data only contains a value for input parameters, there is no targeted value(output). It is easy to collect as compared to the labeled one in the Supervised approach.

Unsupervised Learning Algorithms

There are mainly 3 types of Algorithms which are used for Unsupervised dataset.

- **Clustering**

- **Association Rule Learning**
- **Dimensionality Reduction**

1. Clustering Algorithms

Clustering in unsupervised machine learning is the process of grouping unlabeled data into clusters based on their similarities. The goal of clustering is to identify patterns and relationships in the data without any prior knowledge of the data's meaning.

Broadly this technique is applied to group data based on different patterns, such as similarities or differences, our machine model finds. These algorithms are used to process raw, unclassified data objects into groups. For example, in the above figure, we have not given output parameter values, so this technique will be used to group clients based on the input parameters provided by our data.

Some common clustering algorithms:

- ***K-means Clustering:*** *Groups data into K clusters based on how close the points are to each other.*
- ***Hierarchical Clustering:*** *Creates clusters by building a tree step-by-step, either merging or splitting groups.*
- ***Density-Based Clustering (DBSCAN):*** *Finds clusters in dense areas and treats scattered points as noise.*
- ***Mean-Shift Clustering:*** *Discovers clusters by moving points toward the most crowded areas.*
- ***Spectral Clustering:*** *Groups data by analyzing connections between points using graphs.*

2. Association Rule Learning

Association rule learning is also known as association rule mining is a common technique used to discover associations in unsupervised machine learning. This technique is a rule-based ML technique that finds out some very useful relations between parameters of a large data set. This technique is basically used for market basket analysis that helps to better understand the relationship between different products.

For e.g. shopping stores use algorithms based on this technique to find out the relationship between the sale of one product w.r.t to another's sales based on customer behavior. **Like if a customer buys milk, then he may also buy bread, eggs, or butter**. Once trained well, such models can be used to increase their sales by planning different offers.

Some common Association Rule Learning algorithms:

- ***Apriori Algorithm:*** *Finds patterns by exploring frequent item combinations step-by-step.*
- ***FP-Growth Algorithm:*** *An Efficient Alternative to Apriori. It quickly identifies frequent patterns without generating candidate sets.*
- ***Eclat Algorithm:*** *Uses intersections of itemsets to efficiently find frequent patterns.*
- ***Efficient Tree-based Algorithms:*** *Scales to handle large datasets by organizing data in tree structures.*

3. Dimensionality Reduction

Dimensionality reduction is the process of reducing the number of features in a dataset while preserving as much information as possible. This technique is useful for improving the performance of machine learning algorithms and for data visualization.

Imagine a dataset of 100 features about students (height, weight, grades, etc.). To focus on key traits, you reduce it to just 2 features: height and grades, making it easier to visualize or analyze the data.

*Here are some popular **Dimensionality Reduction algorithms**:*

- ***Principal Component Analysis (PCA):** Reduces dimensions by transforming data into uncorrelated principal components.*
- ***Linear Discriminant Analysis (LDA):** Reduces dimensions while maximizing class separability for classification tasks.*
- ***Non-negative Matrix Factorization (NMF):** Breaks data into non-negative parts to simplify representation.*
- ***Locally Linear Embedding (LLE):** Reduces dimensions while preserving the relationships between nearby points.*
- ***Isomap:** Captures global data structure by preserving distances along a manifold.*

Challenges of Unsupervised Learning

Here are the key challenges of unsupervised learning:

- **Noisy Data**: Outliers and noise can distort patterns and reduce the effectiveness of algorithms.
- **Assumption Dependence**: Algorithms often rely on assumptions (e.g., cluster shapes), which may not match the actual data structure.
- **Overfitting Risk**: Overfitting can occur when models capture noise instead of meaningful patterns in the data.
- **Limited Guidance**: The absence of labels restricts the ability to guide the algorithm toward specific outcomes.
- **Cluster Interpretability**: Results, such as clusters, may lack clear meaning or alignment with real-world categories.
- **Sensitivity to Parameters**: Many algorithms require careful tuning of hyperparameters, such as the number of clusters in k-means.
- **Lack of Ground Truth**: Unsupervised learning lacks labeled data, making it difficult to evaluate the accuracy of results.

Applications of Unsupervised learning

Unsupervised learning has diverse applications across industries and domains. Key applications include:

- **Customer Segmentation:** Algorithms cluster customers based on purchasing behavior or demographics, enabling targeted marketing strategies.

- **Anomaly Detection:** Identifies unusual patterns in data, aiding fraud detection, cybersecurity, and equipment failure prevention.
- **Recommendation Systems**: Suggests products, movies, or music by analyzing user behavior and preferences.
- **Image and Text Clustering**: Groups similar images or documents for tasks like organization, classification, or content recommendation.
- **Social Network Analysis**: Detects communities or trends in user interactions on social media platforms.
- **Astronomy and Climate Science:** Classifies galaxies or groups weather patterns to support scientific research.

STOCHASTIC GRADIENT DESCENT

Stochastic Gradient Descent (SGD) is a variant of the Gradient Descent algorithm that is used for optimizing machine learning models. It addresses the computational inefficiency of traditional Gradient Descent methods when dealing with large datasets in machine learning projects.

In SGD, instead of using the entire dataset for each iteration, only a single random training example (or a small batch) is selected to calculate the gradient and update the model parameters. This random selection introduces randomness into the optimization process, hence the term “stochastic” in stochastic Gradient Descent

The advantage of using SGD is its computational efficiency, especially when dealing with large datasets. By using a single example or a small batch, the computational cost per iteration is significantly reduced compared to traditional Gradient Descent methods that require processing the entire dataset.

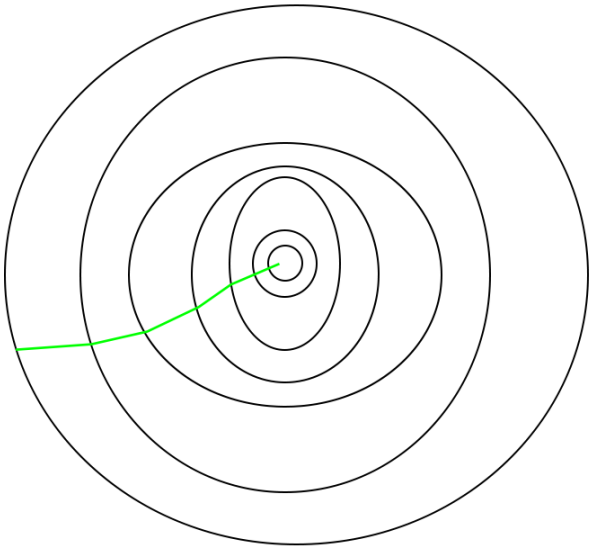
Stochastic Gradient Descent Algorithm

- Initialization: Randomly initialize the parameters of the model.
- Set Parameters: Determine the number of iterations and the learning rate (alpha) for updating the parameters.
- Stochastic Gradient Descent Loop: Repeat the following steps until the model converges or reaches the maximum number of iterations:
    - Shuffle the training dataset to introduce randomness.
    - Iterate over each training example (or a small batch) in the shuffled order.
    - Compute the gradient of the cost function with respect to the model parameters using the current training
      example (or batch).
    - Update the model parameters by taking a step in the direction of the negative gradient, scaled by the learning rate.
    - Evaluate the convergence criteria, such as the difference in the cost function between iterations of the gradient.

- Return Optimized Parameters: Once the convergence criteria are met or the maximum number of iterations is reached, return the optimized model parameters.

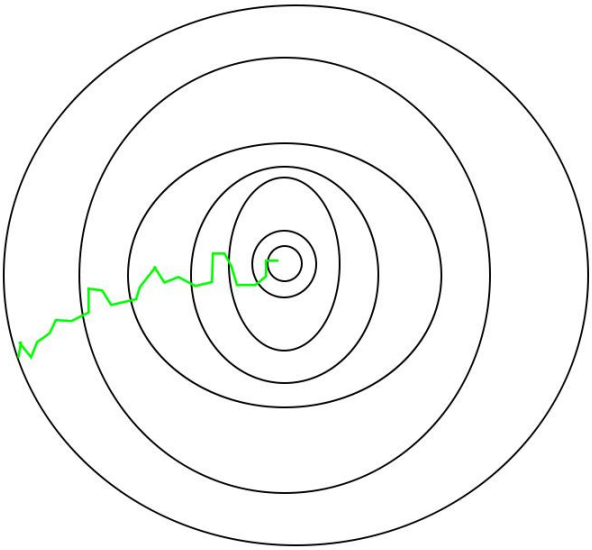
In SGD, since only one sample from the dataset is chosen at random for each iteration, the path taken by the algorithm to reach the minima is usually noisier than your typical Gradient Descent algorithm. But that doesn't matter all that much because the path taken by the algorithm does not matter, as long as we reach the minimum and with a significantly shorter training time.

The path taken by Batch Gradient Descent is shown below:



Batch gradient optimization path

A path taken by Stochastic Gradient Descent looks as follows –



stochastic gradient optimization path

One thing to be noted is that, as SGD is generally noisier than typical Gradient Descent, it usually took a higher number of iterations to reach the minima, because of the randomness in its descent. Even though it requires a higher number of iterations to reach the minima than typical Gradient Descent, it is still computationally much less expensive than typical Gradient Descent. Hence, in most scenarios, SGD is preferred over Batch Gradient Descent for optimizing a learning algorithm.

Difference between Stochastic Gradient Descent & batch Gradient Descent

The comparison between Stochastic Gradient Descent (SGD) and Batch Gradient Descent are as follows:

Aspect	**Stochastic Gradient Descent (SGD)**	**Batch Gradient Descent**
Dataset Usage	Uses a single random sample or a small batch of samples at each iteration.	Uses the entire dataset (batch) at each iteration.
Computational Efficiency	Computationally less expensive per iteration, as it processes fewer data points.	Computationally more expensive per iteration, as it processes the entire dataset.
Convergence	Faster convergence due to frequent updates.	Slower convergence due to less frequent updates.
Noise in Updates	High noise due to frequent updates with a single or few samples.	Low noise as it updates parameters using all data points.
Stability	Less stable as it may oscillate around the optimal solution.	More stable as it converges smoothly towards the optimum.
Memory Requirement	Requires less memory as it processes fewer data points at a time.	Requires more memory to hold the entire dataset in memory.
Update Frequency	Frequent updates make it suitable for online learning and large datasets.	Less frequent updates make it suitable for smaller datasets.

Aspect	Stochastic Gradient Descent (SGD)	Batch Gradient Descent
Initialization Sensitivity	Less sensitive to initial parameter values due to frequent updates.	More sensitive to initial parameter values.

STEPS TO BUILD A MACHINE LEARNING MODEL

Step 1: Data Collection for Machine Learning

Data collection is a crucial step in the creation of a machine learning model, as it lays the foundation for building accurate models. In this phase of machine learning model development, relevant data is gathered from various sources to train the machine learning model and enable it to make accurate predictions. The first step in data collection is defining the problem and understanding the requirements of the machine learning project. This usually involves determining the type of data we need for our project like structured or unstructured data, and identifying potential sources for gathering data.

Once the requirements are finalized, data can be collected from a variety of sources such as databases, APIs, web scraping, and manual data entry. It is crucial to ensure that the collected data is both relevant and accurate, as the quality of the data directly impacts the generalization ability of our machine learning model. In other words, the better the quality of the data, the better the performance and reliability of our model in making predictions or decisions.

Step 2: Data Preprocessing and Cleaning

Preprocessing and preparing data is an important step that involves transforming raw data into a format that is suitable for training and testing for our models. This phase aims to clean i.e. remove null values, and garbage values, and normalize and preprocess the data to achieve greater accuracy and performance of our machine learning models.

As Clive Humby said, **"Data is the new oil. It's valuable, but if unrefined it cannot be used."** This quote emphasizes the importance of refining data before using it for analysis or modeling. Just like oil needs to be refined to unlock its full potential, raw data must undergo preprocessing to enable its effective utilization in ML tasks. The preprocessing process typically involves several steps, including handling missing values, encoding categorical variables i.e. converting into numerical, scaling numerical features, and feature engineering. This ensures that the model's performance is optimized and also our model can generalize well to unseen data and finally get accurate predictions.

Step 3: Selecting the Right Machine Learning Model

Selecting the right **machine learning model** plays a pivotal role in building of successful model, with the presence of numerous algorithms and techniques available easily, choosing the most suitable model for a given problem significantly impacts the accuracy and performance of the model.
The process of selecting the right machine learning model involves several considerations, some of which are:

Firstly, understanding the nature of the problem is an essential step, as our model nature can be of any type like classification, regression, clustering or more, different types of problems require different algorithms to make a predictive model.

Secondly, familiarizing yourself with a variety of machine learning algorithms suitable for your problem type is crucial. Evaluate the complexity of each algorithm and its interpretability. We can also explore more complex models like deep learning may help in increasing your model performance but are complex to interpret.

Step 4: Training Your Machine Learning Model

In this phase of building a machine learning model, we have all the necessary ingredients to train our model effectively. This involves utilizing our prepared data to teach the model to recognize patterns and make predictions based on the input features. During the training process, we begin by feeding the preprocessed data into the selected machine-learning algorithm. The algorithm then iteratively adjusts its internal parameters to minimize the difference between its predictions and the actual target values in the training data. This optimization process often employs techniques like gradient descent.

As the model learns from the training data, it gradually improves its ability to generalize to new or unseen data. This iterative learning process enables the model to become more adept at making accurate predictions across a wide range of scenarios.

Step 5: Evaluating Model Performance

Once you have trained your model, it's time to assess its performance. There are various metrics used to evaluate model performance, categorized based on the type of task: regression/numerical or classification.

For regression tasks, common evaluation metrics are:

- **Mean Absolute Error (MAE):** MAE is the average of the absolute differences between predicted and actual values.
- **Mean Squared Error (MSE):** MSE is the average of the squared differences between predicted and actual values.
- **Root Mean Squared Error (RMSE):** It is a square root of the MSE, providing a measure of the average magnitude of error.
- **R-squared (R2):** It is the proportion of the variance in the dependent variable that is predictable from the independent variables.

For classification tasks, common evaluation metrics are:

- **Accuracy:** Proportion of correctly classified instances out of the total instances.
- **Precision:** Proportion of true positive predictions among all positive predictions.
- **Recall:** Proportion of true positive predictions among all actual positive instances.
- **F1-score:** Harmonic mean of precision and recall, providing a balanced measure of model performance.
- **Area Under the Receiver Operating Characteristic curve (AUC-ROC):** Measure of the model's ability to distinguish between classes.

- **Confusion Metrics:** It is a matrix that summarizes the performance of a classification model, showing counts of true positives, true negatives, false positives, and false negatives instances.

Step 6: Tuning and Optimizing Your Model

As we have trained our model, our next step is to optimize our model more. Tuning and optimizing helps our model to maximize its performance and generalization ability. This process involves fine-tuning hyperparameters, selecting the best algorithm, and improving features through feature engineering techniques. Hyperparameters are parameters that are set before the training process begins and control the behavior of the machine learning model. These are like learning rate, regularization and parameters of the model should be carefully adjusted.

Techniques like grid search cv randomized search and cross-validation are some optimization techniques that are used to systematically explore the hyperparameter space and identify the best combination of hyperparameters for the model. Overall, tuning and optimizing the model involves a combination of careful speculation of parameters, feature engineering, and other techniques to create a highly generalized model.

Step 7: Deploying the Model and Making Predictions

Deploying the model and making predictions is the final stage in the journey of creating an ML model. Once a model has been trained and optimized, it's to integrate it into a production environment where it can provide real-time predictions on new data.

During model deployment, it's essential to ensure that the system can handle high user loads, operate smoothly without crashes, and be easily updated. Tools like Docker and Kubernetes help make this process easier by packaging the model in a way that makes it easy to run on different computers and manage efficiently. Once deployment is done our model is ready to predict new data, which involves feeding unseen data into the deployed model to enable real-time decision making.

CHALLENGES MOTIVATING DEEP LEARNING

Deep learning has emerged as a powerful solution to many AI problems due to the limitations of conventional machine learning algorithms. The primary challenges that motivated deep learning include:

1. Curse of Dimensionality

- As the number of variables increases, the number of possible configurations grows exponentially.
- This poses a statistical challenge: for instance, tracking a single variable requires 10 regions, but tracking two variables requires 100, and three variables require 1000.
- Traditional machine learning struggles with such high-dimensional data, whereas deep learning can model complex relationships effectively.

2. Failure of Traditional ML in AI Problems

- Simple machine learning algorithms work well on many tasks but fail in crucial AI problems like speech recognition and object detection.

- Deep learning was introduced to improve generalization in such cases, as conventional algorithms failed to scale effectively.

3. Local Constancy and Smoothness Regularization

- Traditional ML assumes that the function to be learned does not change significantly in small regions (local constancy).
- While useful, this assumption alone is insufficient for complex tasks.
- Deep learning introduces additional priors beyond smoothness to reduce generalization errors.

4. Kernel Machines and Smoothness Limitations

- Kernel methods work by comparing test samples to training samples based on similarity.
- However, these methods rely too much on template matching, which limits their scalability.
- Deep learning overcomes this by learning hierarchical representations.

5. Decision Trees and Overfitting

- Decision trees divide the input space into multiple regions and use a separate parameter for each region.
- The number of required training samples increases rapidly, leading to poor scalability.
- Deep learning enables parameter sharing across regions, reducing the need for excessive data.

6. Need for More Regions than Examples

- Many ML methods require an equal or greater number of training samples than the number of regions they define.
- Deep learning allows efficient representation of complex functions with fewer samples.

7. Manifold Learning

- The real-world data distribution is often confined to lower-dimensional manifolds embedded in high-dimensional space.
- Deep learning effectively captures these structures, enabling better generalization.
- This is particularly important in image and text processing, where meaningful data occupies only a small fraction of the input space.

8. Manifold Hypothesis for Images and Text

- Images and text data reside in structured, low-dimensional manifolds.
- Randomly sampled images usually look like noise, and random text generation rarely produces meaningful content.
- Deep learning captures these manifolds better than conventional methods.